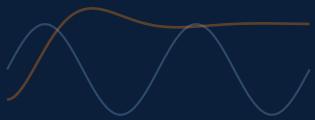
WEEK 1 · DAY 4 / 5

Mechatronics and Robotics Engineering (50 h)



Control (cont.) and Servo Systems I

2.7 Filters & compensation · 2.8 Digital control · 2.9 Control of robot manipulators · 3.1 Servo architecture & modes · 3.2 Power electronics for drives

Mechatronics and Robotics Engineering Programme

Week 1 · Day 4 ▪ 2 weeks · 10 days · 50 hours

Agenda and learning outcomes

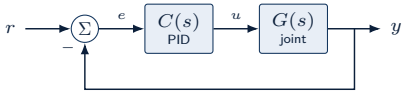
Code	Session	Time
2.7	Filters and compensation	60 min
2.8	Digital control	60 min
2.9	Control of robot manipulators	60 min
3.1	Servo architecture and control modes	60 min
3.2	Power electronics for drives	60 min
	Net teaching time	**5 h**

Short breaks (≈10 min) after each 1-h block are extra.

By the end of today you can . . .

- **calculate** the phase a filter or a sampling delay costs a loop
- **choose** a notch or lead compensator for a given problem
- **explain** gravity compensation and computed torque for a robot arm
- **sketch** a servo drive with its torque, velocity and position loops
- **calculate** DC-bus voltage, PWM average voltage and braking energy

What we learned so far (W1D3 in one slide)



- **feedback**: measure y, compare with r, correct
- **PID**: P reacts to the error now, I to its history, D to its trend
- **anti-windup**: stop the integrator when the output saturates

- **Bode plot**: gain and phase of the loop $L(j\omega) = C\,G$
- **crossover** ω_c: where $|L| = 1$ (0 dB) $\approx$ bandwidth
- **phase margin** PM $= 180^\circ + \angle L(j\omega_c)$; aim for 45–60°

Today's thread

Filters, sampling delay and resonances **cost phase**; lead compensators and fast sampling **buy it back**. Then we open the box: the servo drive that runs these loops.

Session 1

2.7 Filters and Compensation

Refresher: what does a filter do?

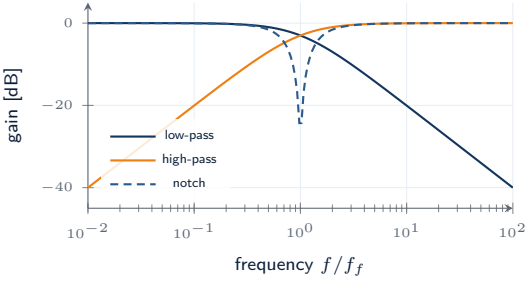
Refresher: a filter is a music equaliser

A **filter** passes some frequencies and weakens others.

- **low-pass** (LPF): treble down, bass kept
- **high-pass**: bass down (removes offsets)
- **notch**: removes one tone (a hum)

$$|F(f)| = \frac{1}{\sqrt{1+(f/f_f)^2}}, \qquad f_f = \frac{1}{2\pi RC}\ [\text{Hz}]$$

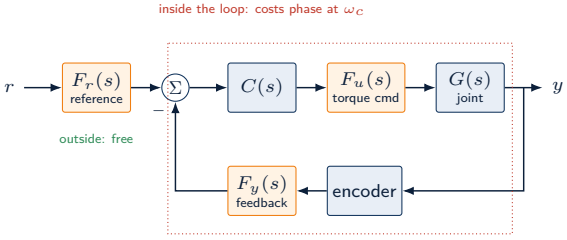
RC low-pass: $R = 1\,\text{k}\Omega$, $C = 1\,\mu\text{F} \Rightarrow f_f = 159\,\text{Hz}$; at 1.59 kHz only 10 % passes.



Take-away

Every filter that weakens a signal also **delays** it a little. Inside a control loop that delay is a cost.

Why filters in a robot joint loop — and where they sit



What we filter against

- encoder quantisation noise ("steps")
- resonances of gearboxes, belts, long links
- jerky set-points from the robot program
- offsets of analogue sensors

Rule

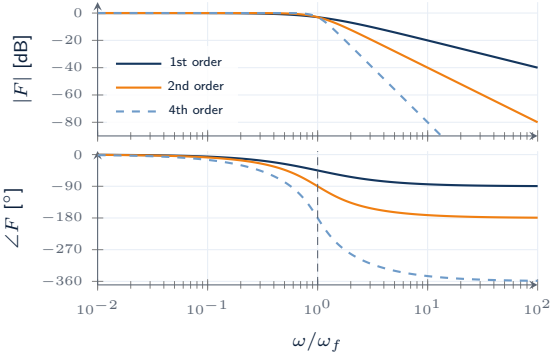
A filter **inside** the loop (F_u, F_y) becomes part of $L(s)$ and eats phase margin. A filter on the reference (F_r) only shapes the motion — no stability cost.

Low-pass filter: cut-off, order and phase

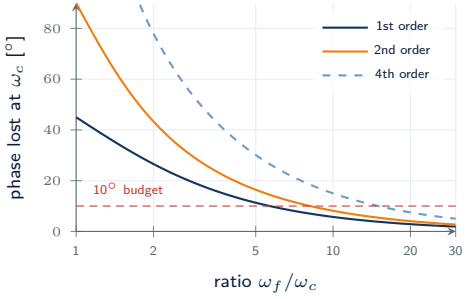
$$F_1(s) = \frac{\omega_f}{s + \omega_f}$$

- **cut-off** ω_f [rad/s]: the -3 dB point
- **order** n: roll-off $-20n$ dB per decade
- phase: $-45°$ per order at ω_f
- 2nd order with low damping **peaks** near ω_f

Take-away: higher order = cleaner signal, but much more phase lag.



The price of filtering: phase lost at crossover



$$\Delta\varphi = \arctan\frac{\omega_c}{\omega_f} \quad \text{(1st-order LPF)}$$

Worked example. Joint velocity loop, $f_c = 150\,\text{Hz}$; we allow 10° of phase loss.

1. need $f_c/f_f \le \tan 10^\circ = 0.176$
2. $f_f \ge 150\,\text{Hz}/0.176 = 850\,\text{Hz}$
3. check: $\arctan(150/850) = 10.0^\circ$ ✓

Trade-off

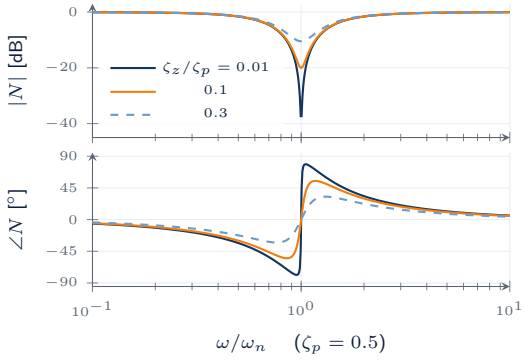
Lower ω_f = quieter motor but less phase margin. Rule of thumb: $\omega_f \approx (5\ldots10)\,\omega_c$.

Notch filter: cutting out one frequency

Picture: a playground swing has one natural frequency — so does a robot joint with a springy gearbox. A **notch** stops the controller "pushing" at exactly that frequency.

$$N(s) = \frac{s^2 + 2\zeta_z \omega_n s + \omega_n^2}{s^2 + 2\zeta_p \omega_n s + \omega_n^2}$$

- **centre** ω_n: set to the resonance
- **depth**: ratio ζ_z/ζ_p ($0.1 \to -20\,\text{dB}$)
- **width**: set by ζ_p; wide = robust, more lag

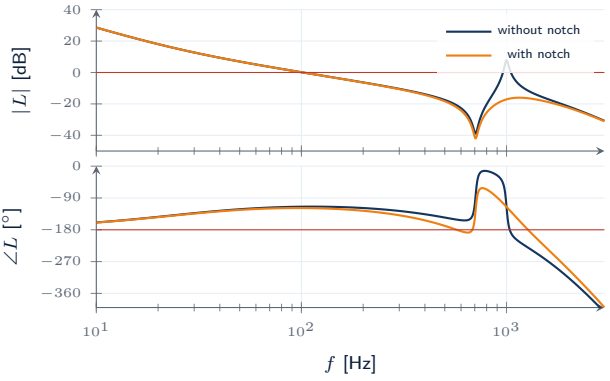


Notch in practice: taming a 1 kHz resonance

Axis: motor – stiff coupling – load, resonance at $f_r = 1\,\text{kHz}$. Velocity PI with $f_c = 100\,\text{Hz}$.

	no notch	notch
0 dB crossings	3	1
PM at 100 Hz	65°	61°
closed loop	unstable	stable

The resonance lifts $|L|$ above 0 dB near 1 kHz with no phase left. A $-25\,\text{dB}$ notch removes the peak and costs only $\approx 4^\circ$ at 100 Hz.



Model and numbers computed for this course; cf. [Ellis, 2012, ch. 16]. Joint resonances return in W2D1 (session 3.7).

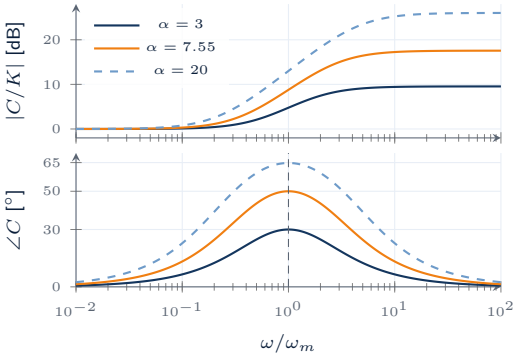
Phase-lead compensator: buying phase back

$$C_{\text{lead}}(s) = K\,\frac{1+\alpha T s}{1+Ts}, \qquad \alpha > 1$$

- a zero before a pole: phase goes **up** between them
- maximum lead at $\omega_m = 1/(T\sqrt{\alpha})$ — put it at ω_c
- $\alpha = 3 \to 30^\circ$, $\alpha = 7.5 \to 50^\circ$, $\alpha = 20 \to 65^\circ$
- price: high-frequency gain $\times\alpha \Rightarrow$ more noise

Picture: steering a bicycle by looking ahead — the lead reacts to where the error is *going*.

[Ogata, 2010, ch. 7]; [Franklin, Powell & Emami-Naeini, 2019, ch. 6].

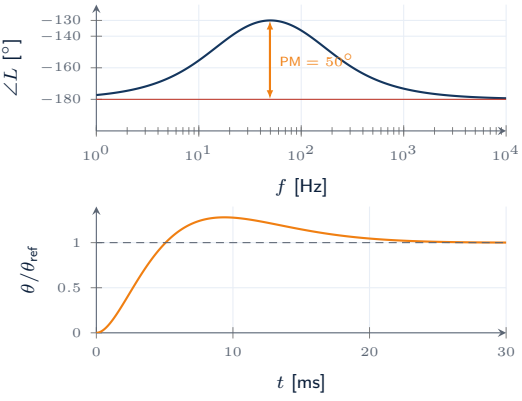


Lead design — worked example: position loop on a pure inertia

Plant $G = 1/(Js^2)$, $J = 2 \times 10^{-4}\,\text{kg m}^2$: phase is $-180°$ everywhere, so PM $= 0$ with a plain gain.
Goal: $f_c = 50\,\text{Hz}$ ($\omega_c = 314\,\text{rad/s}$), PM $= 50°$.

1. need $50°$ of lead $\Rightarrow \alpha = 7.55$ (graph)
2. centre it: $T = 1/(\omega_c\sqrt{\alpha}) = 1.16\,\text{ms}$
3. gain for $|L| = 1$ at ω_c: $K = J\omega_c^2/\sqrt{\alpha} = 7.2\,\text{N m/rad}$
4. check (computed): $f_c = 49.5\,\text{Hz}$, PM $= 50°$ ✓

Step overshoot 28 % (from the lead zero); a reference filter F_r can remove it.

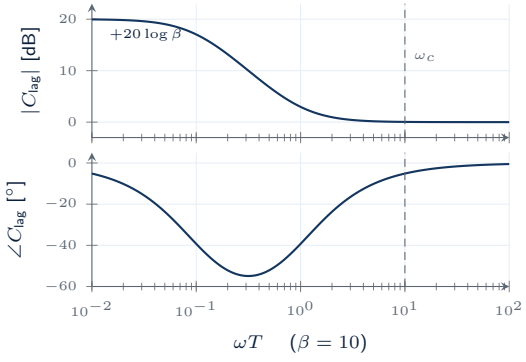


Phase-lag compensator and lead–lag

$$C_{\text{lag}}(s) = \beta\,\frac{1+Ts}{1+\beta Ts}, \qquad \beta > 1$$

- boosts gain at **low** frequency by $\beta \Rightarrow$ smaller steady error, stiffer against load
- leaves high frequencies (and noise) alone
- zero a decade below ω_c: costs only $\approx 5^\circ$
- **lead–lag**: lead buys PM, lag buys stiffness

PID is a lead–lag in disguise: PI $\approx$ lag, filtered D $\approx$ lead.



Going further: a **high-pass** filter $s/(s+\omega_h)$ removes offsets (e.g. accelerometer drift) but must never sit in the main position feedback path.

Session 2.7 — Check your understanding

Check your understanding

1. A colleague adds a 4th-order low-pass at $2\,f_c$ to the joint velocity feedback "to clean it up". What happens to the phase margin?
2. Why does a reference filter F_r not change stability, while a feedback filter F_y does?
3. The robot joint whistles at 800 Hz when you raise the gain. Which filter do you reach for, and where do you set its centre?

Discuss in pairs for 2 minutes, then we compare answers.

Session 2.7 — Summary and key terms

Key take-aways

- In-loop filters are part of $L(s)$: budget their phase ($\omega_f \approx 5$–$10\,\omega_c$).
- A notch removes one resonance for a few degrees of PM.
- Lead adds phase at ω_c; lag adds low-frequency stiffness.
- Reference filters are "free": they shape motion, not stability.

Key terms

cut-off -3 dB frequency · **order** steepness of roll-off · **low/high-pass** keeps slow/fast parts · **notch** removes one frequency · **resonance** frequency where a structure swings strongly · **lead/lag** compensator adding phase / low-frequency gain

Session 2

2.8 Digital Control

Refresher: sampling a signal

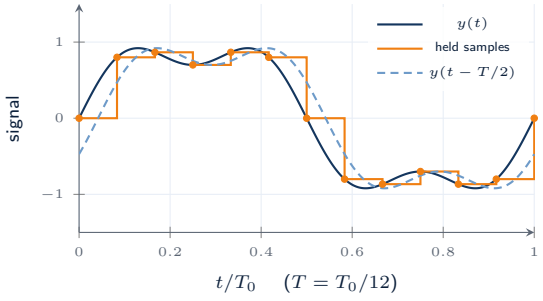
Refresher: sampling is taking snapshots

A film shows motion as 24 photos per second. A digital controller sees a signal only at the instants $t = kT$:

$$f_s = \frac{1}{T} \quad [\text{Hz}], \qquad y_k = y(kT)$$

Between samples the output is **held** constant: the **zero-order hold** (ZOH) makes a staircase.

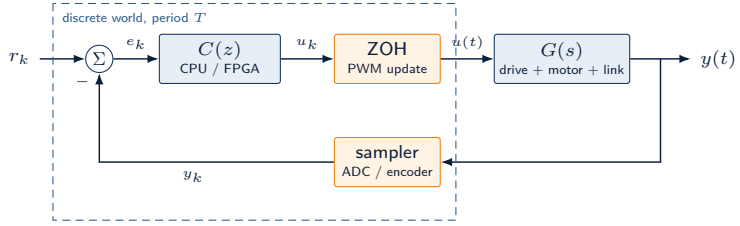
Example: $f_s = 8\,\text{kHz} \Rightarrow T = 125\,\mu\text{s}$; a 100 Hz sine gets 80 samples per period.



Take-away

The staircase follows the signal **half a sample late** on average: holding costs a delay of $T/2$.

Anatomy of a digital joint controller



- the computer sees y only at $t = kT$
- the output is held until the next update
- computing takes time: u_k is usually applied **one sample later**

Loop	typical f_s	typical f_c
current	8–32 kHz	1–3 kHz
velocity	4–16 kHz	100–500 Hz
position	1–8 kHz	20–100 Hz
robot path planner	0.25–1 kHz	—

Delay budget and phase loss at crossover

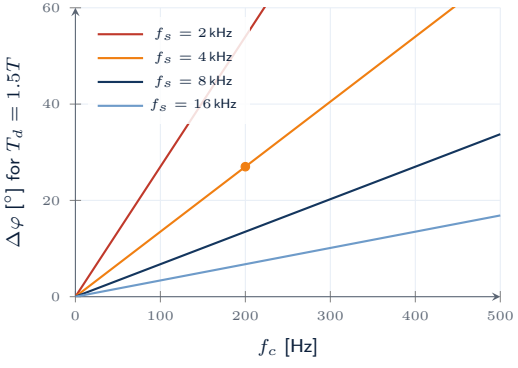
Picture: a shower with a long pipe — you turn the tap, the temperature changes later, and you overcorrect.

$$\Delta\varphi = 360^\circ \cdot f_c \cdot T_d, \qquad T_d \approx 1.5\,T$$

($T/2$ from the hold + T for computing.)

Worked example: $f_s = 4\,\text{kHz}$, $f_c = 200\,\text{Hz}$.

1. $T = 1/4000 = 250\,\mu\text{s}$
2. $T_d = 1.5 \times 250 = 375\,\mu\text{s}$
3. $\Delta\varphi = 360 \cdot 200 \cdot 0.000375 = \mathbf{27^\circ}$
4. at 8 kHz: 13.5° — double f_s, halve the loss



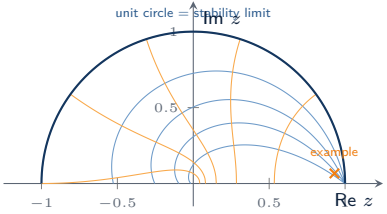
From the s-plane to the z-plane

In discrete time a pole is a number z that multiplies the signal every sample: $x_{k+1} = z\,x_k$.

$$z = e^{sT} \qquad \text{stable} \iff |z| < 1$$

- $|z| < 1$: each sample smaller $\Rightarrow$ decays
- left half s-plane $\rightarrow$ inside the unit circle
- near $z = 1$: slow; near $z = -1$: rings at $f_s/2$

Example: $s = -70 \pm j71$, $T = 1\,\text{ms}$
$\Rightarrow z \approx 0.93 \pm j0.07$: inside the circle, stable.



thin lines: constant damping (blue-grey) and constant frequency (amber)

Discrete PID: a few lines every sample

Every T seconds the controller runs:

- **I**: add up the error, $I_k = I_{k-1} + K_i T\, e_k$
- **D**: difference of the measurement, filtered
- **saturation**: output limited (max. current)
- **anti-windup**: when saturated, stop the integrator growing

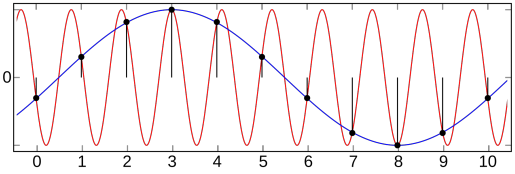
Picture: the integral is a bucket that collects error; the derivative is the slope between two snapshots.

```
// executed every T (interrupt synced to PWM)
e  = r - y;
I += Ki*T*e;                      // integrate
D  = a*D - b*(y - y_old);         // filtered D on y
y_old = y;                 // a=Tf/(Tf+T), b=Kd/(Tf+T)
u  = Kp*e + I + D;
if (u > UMAX)      { I -= u - UMAX; u = UMAX; }
else if (u < -UMAX){ I -= u + UMAX; u = -UMAX;}
write_pwm(u);        // takes effect next period
```

D acts on $-y$ so a step in r does not cause a "kick".

Going further: discretisation methods (forward/backward Euler, Tustin, ZOH equivalent) and frequency pre-warping [Franklin, Powell & Workman, 1998, ch. 6].

Aliasing: the wagon-wheel effect



Moxfyre, CC BY-SA 3.0, via Wikimedia Commons

red: $f = 0.9$, blue: $f = 0.1$, sampled at $f_s = 1$ — identical samples

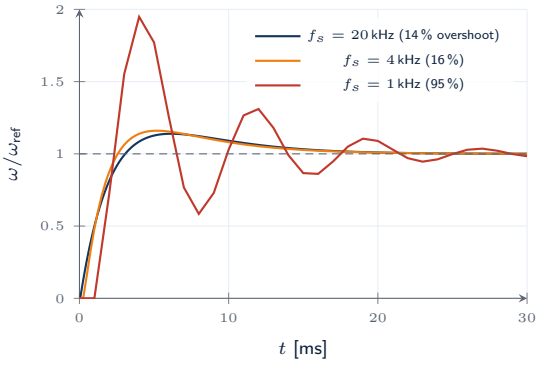
Picture: in films a fast wheel seems to turn slowly or backwards — the camera samples too slowly.

$$f_{\text{alias}} = |f - k\,f_s|\,, \quad k = \text{round}(f/f_s)$$

- anything above $f_s/2$ folds back below it
- 4.6 kHz vibration sampled at 4 kHz looks like **600 Hz**
- cure: filter *before* sampling (anti-alias filter)

More in W2D5 (session 6.5).

Choosing the sample rate



[Franklin, Powell & Workman, 1998, ch. 11]; simulation for this course.

Same velocity PI ($f_c = 100\,\text{Hz}$), only f_s changes.

At 1 kHz: $T_d = 1.5$ ms
$\Rightarrow \Delta\varphi = 360 \cdot 100 \cdot 0.0015 = 54^\circ \Rightarrow$ almost unstable.

Rules of thumb

- $f_s \geq 20$–$40\times$ the loop bandwidth
- check $360^\circ f_c T_d$ against the PM budget
- inner loops sample faster

Session 2.8 — Check your understanding

Check your understanding

1. A current loop crosses at 2 kHz and is sampled at 16 kHz. How much phase does the $1.5T$ delay cost?
2. A discrete pole sits at $z = 1.05$. What does the signal do from one sample to the next?
3. Why does the integrator "wind up" when the drive hits its current limit?

Discuss in pairs for 2 minutes, then we compare answers.

Session 2.8 — Summary and key terms

Key take-aways

- Sampling + hold + computing $\approx 1.5T$ delay.
- Phase lost: $360^\circ f_c T_d$ — double f_s, halve it.
- Discrete stability: all poles inside the unit circle.
- Discrete PID needs saturation and anti-windup.
- Filter before sampling to avoid aliasing.

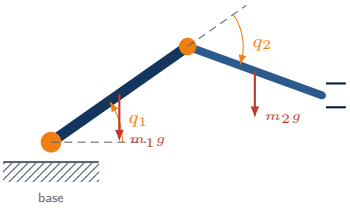
Key terms

sample rate $f_s = 1/T$ · **ZOH** holds the output between samples · z-**plane** map of discrete poles · **anti-windup** stops the integrator in saturation · **aliasing** fast signal masquerading as slow

Session 3

2.9 Control of Robot Manipulators

Why a robot arm is harder than one motor



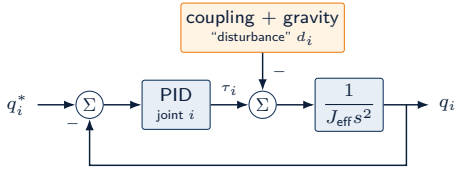
A two-link arm: each joint has its own motor and servo loop, *but*

- **gravity** pulls with a torque that depends on the pose
- the **inertia** seen by joint 1 changes when joint 2 moves
- **coupling**: accelerating one joint pushes on the other
- the payload changes all of the above

$$\boldsymbol{M}(\boldsymbol{q})\,\ddot{\boldsymbol{q}} + \boldsymbol{c}(\boldsymbol{q}, \dot{\boldsymbol{q}}) + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{\tau}$$

inertia · acceleration + Coriolis/centrifugal + gravity = motor torques (details in W2D2, session 4.6)

Independent-joint control: one servo per joint



- every joint is treated as a **SISO** servo (W1D3 tools)
- the other joints' effects are lumped into a disturbance
- standard in industrial robots: simple, robust

Why it works: the gearbox. Load inertia seen by the motor shrinks with N^2 (W1D2):

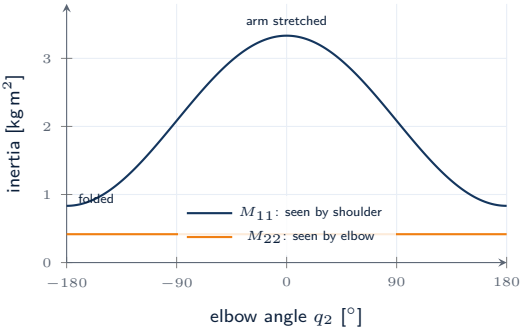
$$J_{\text{eff}} = J_m + \frac{J_{\text{link}}}{N^2}$$

Worked example: $J_m = 3 \times 10^{-4}\,\text{kg m}^2$, link inertia varies 0.8–3.3 kg m^2, $N = 100$.

1. J_{link}/N^2: 0.8×10^{-4} to 3.3×10^{-4}
2. J_{eff}: 3.8×10^{-4} to $6.3 \times 10^{-4}\,\text{kg m}^2$
3. variation only $\times 1.7$ instead of $\times 4$ ✓

Take-away: high gear ratios hide the coupling — direct drives do not.

The inertia a joint feels changes with the pose



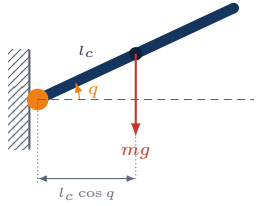
two links, $m = 5\,\text{kg}$, $l = 0.5\,\text{m}$ each; computed for this course

Picture: spin on an office chair with arms stretched out, then pull them in — you speed up. Same mass, different inertia.

- stretched arm: shoulder inertia $\approx 3.3\,\text{kg m}^2$
- folded arm: $\approx 0.8\,\text{kg m}^2$ — **4× less**
- fixed PID gains: sluggish when stretched, nervous when folded

Remedies: high gear ratio, gain scheduling with q_2, or model-based control (next slides).

Gravity torque on a joint



Picture: hold a broom horizontally by one end — hard; hold it straight up — easy. The lever arm of gravity changes with the angle.

$$\tau_g(q) = m\, g\, l_c \cos q$$

m link mass [kg], l_c distance joint–centre of mass [m], q angle from horizontal.

Example: $m = 2$ kg, $l_c = 0.25$ m:

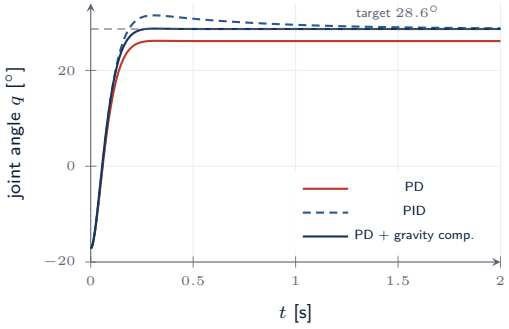
1. horizontal ($q = 0$): $2 \cdot 9.81 \cdot 0.25 = 4.9$ N m
2. at $q = 60^\circ$: $4.9 \cdot 0.5 = 2.45$ N m
3. vertical ($q = 90^\circ$): 0 N m

Gravity compensation: cancel the weight, then control

$$\tau = \underbrace{K_p\, e - K_d\, \dot{q}}_{\text{PD}} + \underbrace{\hat{\tau}_g(q)}_{\text{model}}$$

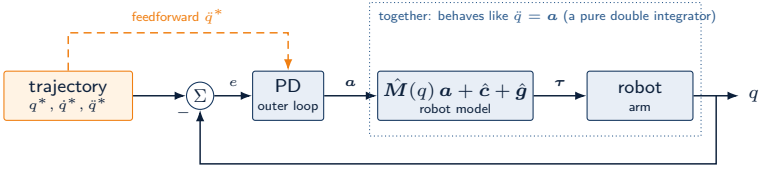
- plain PD: the arm sags until $K_p e = \tau_g$
- sag $e = \tau_g / K_p = 4.4/100 \approx 0.044$ rad (2.5°)
- adding $\hat{\tau}_g(q)$ from the model: **zero sag**, no waiting
- PID also removes the sag, but slowly and with overshoot

Picture: a counterweight on a desk lamp — the spring carries the weight, your hand only positions it.



$m = 2\,\text{kg}$, $l_c = 0.25\,\text{m}$, $J = 0.2\,\text{kg m}^2$, $K_p = 100\,\text{N m/rad}$; simulated for this course

Computed-torque control (conceptual)

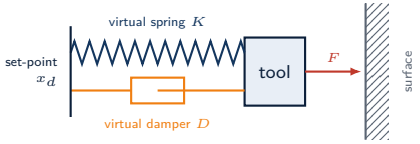


- use the model to **cancel** inertia changes, coupling and gravity
- what is left looks like one simple mass per joint
- a fixed PD then works the same in every pose

Catch: only as good as the model. A wrong payload mass leaves a residual error — feedback still has to clean it up.

Going further: also called inverse-dynamics control or feedback linearisation; robust and adaptive versions estimate the payload online [Spong, Hutchinson & Vidyasagar, 2020, ch. 9; Siciliano et al., 2009, ch. 8].

When the robot touches things: force and impedance control



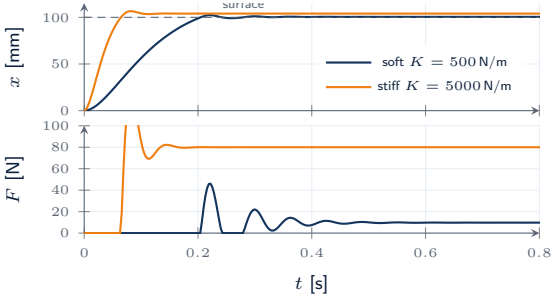
$$F = K\,(x_d - x) - D\,\dot{x}$$

Picture: a handshake — you do not dictate the exact hand position, you stay *springy*.

- **position control**: stiff; a hard contact means huge forces
- **force control**: regulate the contact force directly (polishing, insertion)
- **impedance control**: make the tool behave like a chosen spring + damper

Example: set-point 20 mm behind the surface, $K = 500\,\text{N/m} \Rightarrow F \approx 500 \cdot 0.02 = 10\,\text{N}$ — the stiffness sets the force.

Impedance control: soft versus stiff contact



tool 2 kg, surface stiffness 20 N/mm, set-point 20 mm behind it; simulated

- soft: contact force settles at $\approx 10\,\mathrm{N}$ — gentle
- stiff: $\approx 80\,\mathrm{N}$ and a fast hit
- the force depends on our K, **not** on hitting the exact position
- cobots use low stiffness near people (W2D4, ISO/TS 15066)



GrowSkills Robotics, CC BY-SA 4.0, via Wikimedia Commons (cropped)

Session 2.9 — Check your understanding

Check your understanding

1. An arm under plain PD control sags 2° below the target when held horizontal, but not when pointing straight up. Why?
2. Why is independent-joint PID good enough for a robot with 100:1 gearboxes, but not for a direct-drive arm?
3. A robot must wipe a table with a sponge. Would you use stiff position control or impedance control? Why?

Discuss in pairs for 2 minutes, then we compare answers.

Session 2.9 — Summary and key terms

Key take-aways

- Robot joints are coupled; inertia and gravity change with the pose.
- Independent-joint PID + high gear ratio: the industrial standard.
- Gravity compensation removes sag without waiting for I-action.
- Computed torque cancels the dynamics with a model.
- Impedance control makes contact safe and predictable.

Key terms

independent-joint control one SISO loop per joint · **coupling** motion of one joint loads another · **gravity compensation** feed the modelled weight torque · **computed torque** model-based cancellation of dynamics · **impedance** chosen spring–damper behaviour · **force control** regulate contact force

Session 4

3.1 Servo System Architecture and Control Modes

What is a servo system?

A **servo** (from Latin *servus*, servant) is a motor system that **obeys a command** using feedback: "go to $90°$", "turn at 1000 rpm", "push with 5 N m".

- **servo motor**: usually a PMSM with an encoder and brake
- **servo drive**: power electronics + control loops + protection
- **motion controller**: plans the moves for all axes

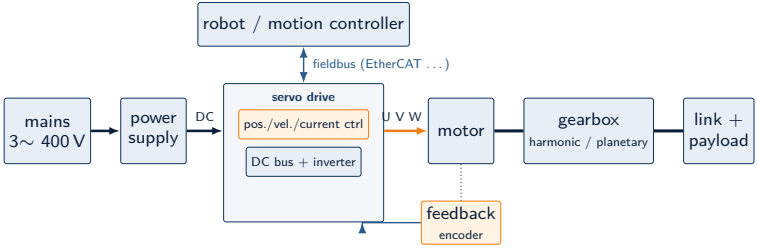
Picture: a taxi — the passenger (controller) says where to go, the driver (drive) handles pedals and steering, the engine (motor) does the work.



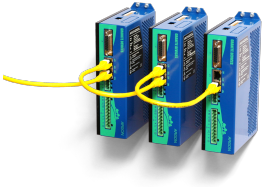
Projuktiponno, CC BY-SA 4.0, via Wikimedia Commons

industrial servo drive and motor

Servo system architecture: from mains to load



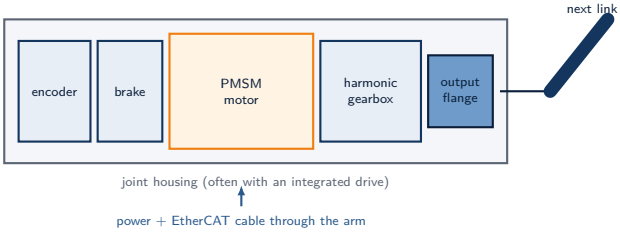
- the **motion controller** sends targets every 0.25–4 ms
- the **drive** closes the fast loops (up to 32 kHz)
- the **encoder** closes every loop



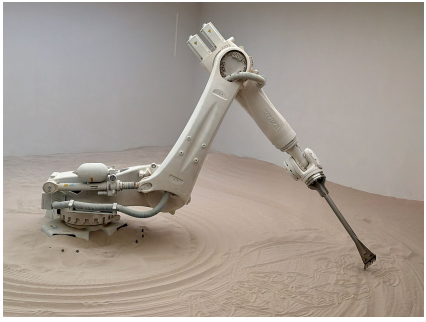
Tero Kontkanen, CC BY-SA 3.0, via Wikimedia Commons

compact servo drives sharing a DC bus and a fieldbus

Inside a robot joint: one servo axis per joint



- a 6-axis robot = **six servo axes** coordinated by one controller
- the **holding brake** keeps the arm up when power is off
- gearbox ratio 50–160 (W1D2)



Oleg Yunakov, CC BY-SA 4.0, via Wikimedia Commons



Pieceofmetalwork, CC BY-SA 4.0, via Wikimedia Commons

Torque, velocity and position control modes

Mode	Controller sends	Loops in the drive	Typ. BW	Robotics example
Torque	torque T^*	current only	1–3 kHz	computed torque, force/impedance control, cobots
Velocity	speed ω^*	velocity + current	100–500 Hz	AMR wheels, conveyor tracking
Position	angle θ^*	position + velocity + current	20–100 Hz	industrial arm joints, gantries

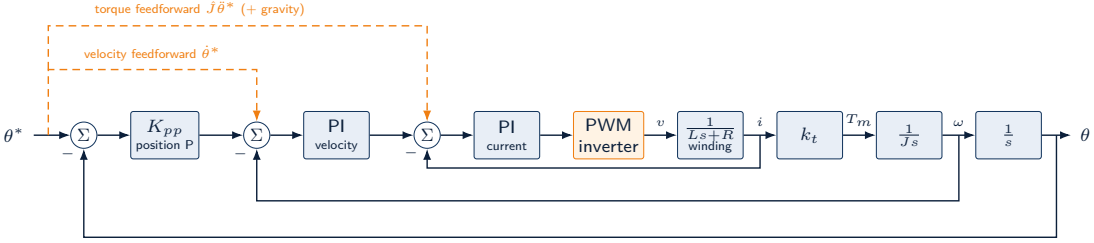
Analogy: telling a driver "press the pedal this much" (torque), "drive at 50 km/h" (velocity) or "stop at that line" (position).

Going further: fieldbus drive profiles (CiA 402) call these CST, CSV, CSP — W2D5.

Selection rule

Close each loop where its **sensor** is and where it can be updated **fast enough**. Model-based robot control (2.9) needs **torque mode**.

Cascade control structure of a servo drive

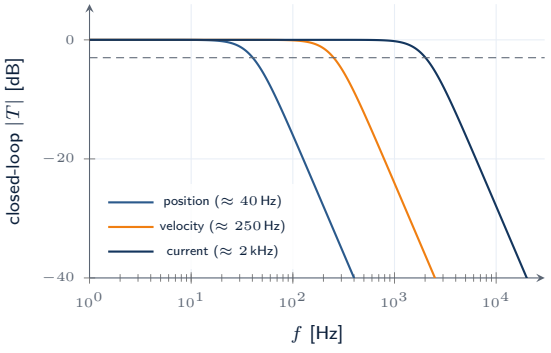


Picture: Russian dolls — each inner loop turns its part into something simple for the next one (current loop → "ideal torque source").

- each inner loop **3–10×** faster than the next
- tune from inside out: current → velocity → position
- feedforward acts on the plan before an error appears

[Ellis, 2012, ch. 17]; details and tuning in W1D5 (sessions 3.4–3.5).

The bandwidth hierarchy



Bandwidth = how fast a loop can follow its command (the -3 dB point).

- current ≈ 2 kHz: limited by PWM and sampling
- velocity ≈ 250 Hz: limited by inertia, resonances, encoder noise
- position ≈ 40 Hz: limited by the velocity loop

Take-away

Separate the loops by **3–10×** so each outer loop sees the inner one as "instant".

Session 3.1 — Check your understanding

Check your understanding

1. A cobot controller computes joint torques from a dynamic model every 1 ms. Which drive mode should the joints run in?
2. The position loop is tuned to 300 Hz while the velocity loop reaches only 100 Hz. What goes wrong?
3. Why must the current loop be tuned first?

Discuss in pairs for 2 minutes, then we compare answers.

Session 3.1 — Summary and key terms

Key take-aways

- Servo = supply + drive + motor + gearbox + load + encoder + controller.
- A robot joint is one complete servo axis.
- Mode = which loops the drive closes (torque / velocity / position).
- Cascade: current $\gg$ velocity $\gg$ position bandwidth.

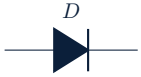
Key terms

servo drive power stage + loops · **motion controller** plans the moves · **control mode** torque, velocity or position · **cascade** loops inside loops · **feedforward** act on the reference · **bandwidth** how fast a loop follows

Session 5

3.2 Power Electronics for Drives

Refresher: diode, capacitor and transistor switch







Diode

A one-way **valve**: current flows only from anode to cathode. Turns AC into DC (rectifier).

Capacitor

An energy **tank**: smooths the voltage and buffers bursts of current.

$$E = \tfrac{1}{2} C V^2$$

Transistor switch

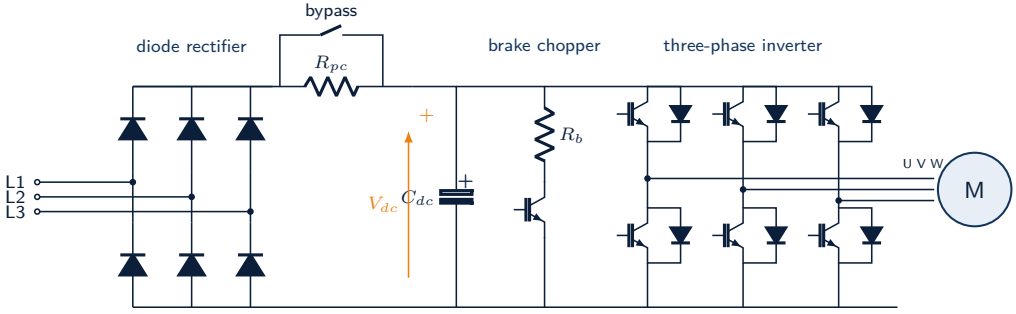
A tap opened by a small **gate** signal. Used only fully **on** or fully **off**.

Why only on/off? Power loss $P = V \cdot I$ in the switch. At $I = 10\,\text{A}$: on: $V \approx 1.5\,\text{V} \Rightarrow 15\,\text{W}$; off: $I \approx 0 \Rightarrow 0\,\text{W}$; half-way ($V = 280\,\text{V}$): **2.8 kW** — it would melt.

Take-away

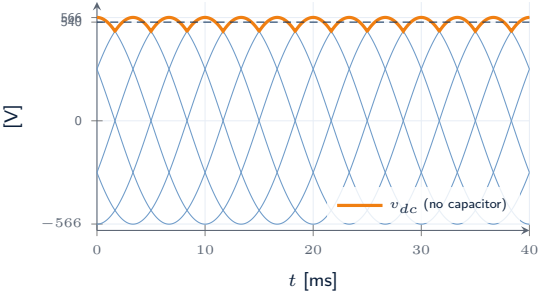
A drive builds any voltage by switching very fast between on and off and averaging (PWM).

Power stage of a servo drive: overview



- **AC** $\rightarrow$ **DC** $\rightarrow$ **AC**: diodes rectify the mains, the capacitor holds a steady **DC bus**
- six transistor switches chop it into motor voltages of any amplitude and frequency
- the **brake chopper** burns surplus energy when the motor brakes

Rectifier and DC bus



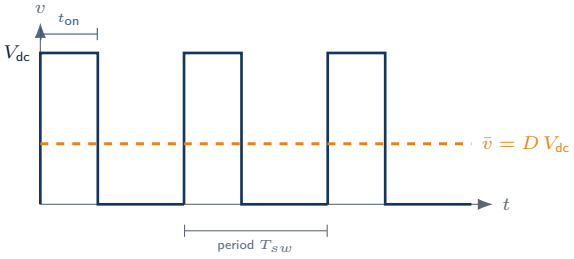
The diodes always pick the highest of the six line-to-line voltages: a bumpy DC.

$$V_{dc} \approx 1.35\, V_{LL} \text{ (average)}, \quad \hat{V} = \sqrt{2}\, V_{LL}$$

Example: 400 V mains: $1.35 \times 400 = 540$ V; with a capacitor it charges to the peak $\sqrt{2} \cdot 400 \approx 566$ V.

- C_{dc} smooths the bumps
- pre-charge resistor limits the inrush into an empty C_{dc}
- diodes cannot send energy **back** to the mains
- small robots and AMRs: a 24–48 V battery *is* the DC bus

PWM: duty cycle and average voltage



Analogy: flick a light switch on and off very fast — the room looks evenly *dimmed*. The motor inductance smooths the current pulses like a flywheel.

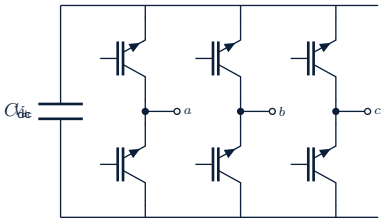
- **PWM** (pulse-width modulation): switch fully on/off at a fixed rate, vary the on-time
- **duty cycle** $D = t_{\text{on}}/T_{sw}$ $(0 \ldots 1)$

$$\bar{v} = D \cdot V_{\text{dc}}$$

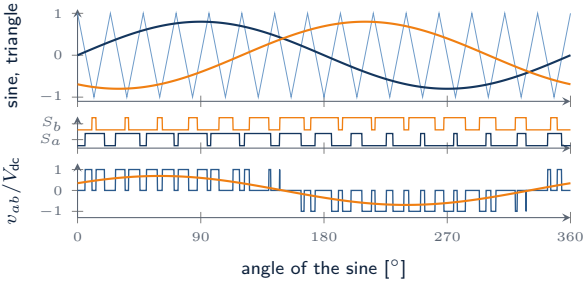
Example: $V_{\text{dc}} = 48\,\text{V}$ (robot joint), $f_{sw} = 20\,\text{kHz}$ ($T_{sw} = 50\,\mu\text{s}$), $t_{\text{on}} = 15\,\mu\text{s}$:

1. $D = 15/50 = 0.3$
2. $\bar{v} = 0.3 \cdot 48 = 14.4\,\text{V}$

Three-phase inverter and sinusoidal PWM

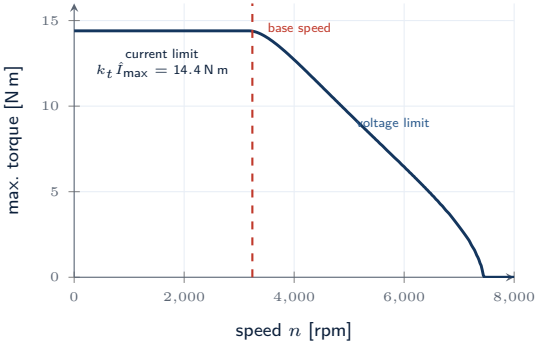


- each **leg**: two switches, **never on together**
- compare a sine with a fast triangle
- sine above triangle ⇒ upper switch on
- duty cycle follows the sine ⇒ average voltage is a sine



peak phase voltage $\hat{V}_1 = m\,\frac{V_{dc}}{2}, \quad m = \underset{(0\ldots1)}{\textbf{modulation index}}$

Voltage and current limits: the torque–speed envelope



computed for a reference servo motor, $V_{\text{dc}} = 325$ V

- **current limit**: switches and windings get hot $\Rightarrow$ maximum torque
- **voltage limit**: back-EMF grows with speed; the inverter gives at most $\approx V_{\text{dc}}/\sqrt{3}$ per phase

$$E = \omega_e\, \psi_{\text{PM}} \;\leq\; V_{\max}$$

Example: $V_{\max} = 325/\sqrt{3} = 188$ V, $\psi_{\text{PM}} = 0.12$ Wb $\Rightarrow \omega_e \leq 188/0.12 \approx 1560$ rad/s (electrical) — with 4 pole pairs ≈ 3730 rpm at no load.

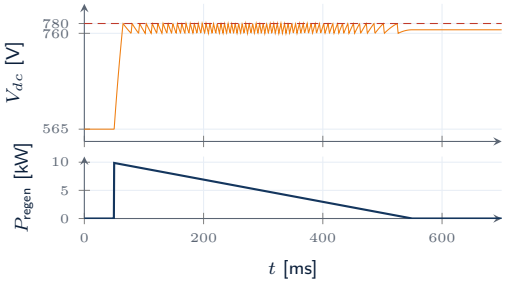
Going further: field weakening extends the speed range (W1D5, session 3.3).

Regeneration and the brake chopper

Picture: a robot arm lowering a heavy part, or braking fast, turns the motor into a **generator** — energy flows back into the DC bus.

Worked example: $J = 0.05\,\text{kg m}^2$ stops from 314 rad/s (3000 rpm) in 0.5 s.

1. energy: $E = \frac{1}{2} J\omega^2 = \frac{1}{2} \cdot 0.05 \cdot 314^2 = 2.47\,\text{kJ}$
2. capacitor (1 mF, 565→780 V) takes only $\frac{1}{2} C(780^2 - 565^2) = 145\,\text{J}$
3. peak power $P = J\alpha\,\omega = 31.4 \cdot 314 = 9.9\,\text{kW}$
4. resistor $R_b \le V^2/P = 780^2/9870 = 62\,\Omega \Rightarrow$ choose **56 Ω**



Chopper on at 780 V, off at 760 V.



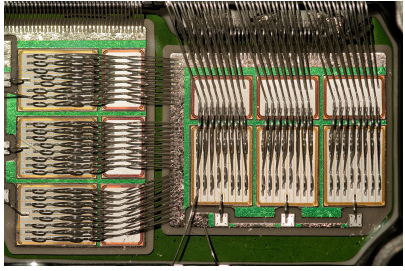
brake resistor: Glogger, CC BY-SA 3.0, via Wikimedia Commons

Power switches and dead time

	Si IGBT	Si MOSFET	SiC MOSFET
voltage	600–1700 V	$\leq$ 250 V	650–1700 V
switching	slower	very fast	very fast
typ. f_{sw}	4–16 kHz	20–100 kHz	16–100 kHz
used in	400 V drives	24–72 V cobots, AMRs	compact drives

- **dead time** t_d (0.5–3 µs): both switches of a leg off at every change, to avoid **shoot-through** (a short circuit of the DC bus)
- price: a small voltage error $\Delta\bar{v} \approx t_d\, f_{sw}\, V_{dc}$
- example: $1\,\mu s \cdot 16\,\text{kHz} \cdot 325\,\text{V} = 5.2\,\text{V}$

[Mohan, Undeland & Robbins, 2003, ch. 8]; [Krishnan, 2001].



Pslawinski, CC BY-SA 3.0, via Wikimedia Commons

opened IGBT module: chips and bond wires

Session 3.2 — Check your understanding

Check your understanding

1. Why can a servo drive not simply use a “half-on” transistor, like a variable resistor, to set the motor voltage?
2. A 48 V joint drive needs an average of 12 V. What duty cycle does it use?
3. A drive trips on DC-bus over-voltage only while the robot lowers a heavy part. What is missing or too small?

Discuss in pairs for 2 minutes, then we compare answers.

Session 3.2 — Summary and key terms

Key take-aways

- Rectifier → DC bus → brake chopper → inverter → motor.
- $V_{dc} \approx 1.35\,V_{LL}$: 540 V at 400 V mains.
- PWM: $\bar{v} = D\,V_{dc}$; the inverter makes sines from it.
- Current limit sets torque; voltage limit sets top speed.
- Braking energy goes into R_b, not the capacitor.

Key terms

DC bus capacitor-backed DC link · **inverter** DC to 3-phase AC · **PWM** on/off switching with variable on-time · **duty cycle** on-time fraction · **regeneration** motor feeding energy back · **brake chopper** switch + resistor that burns it · **shoot-through** both switches of a leg on

Day summary

Control (2.7–2.9)

- filters cost phase; notches remove resonances; lead buys phase back
- sampling delay $\approx 1.5T$ costs $360^\circ f_c T_d$
- robot arms: independent joints + gravity compensation; computed torque and impedance control for demanding tasks

Servo hardware (3.1–3.2)

- drive + motor + gearbox + encoder + controller, one per joint
- torque / velocity / position modes; cascaded loops
- rectifier, DC bus, PWM inverter, brake chopper

One sentence

A robot joint is a servo axis whose bandwidth is limited by phase: filters, delay and resonances spend it; compensators and fast sampling buy it back.

Preview of W1D5: Servo Systems II

- 3.3 Field-oriented control: Clarke/Park, i_d and i_q currents
- 3.4 Cascaded current, velocity and position loops in detail
- 3.5 Loop tuning and bandwidth: inner $\rightarrow$ outer, inertia ratio, delays
- 3.6 Trajectory generation, feedforward and disturbance compensation

Bring with you: the delay formula $360^\circ f_c T_d$, the cascade structure, the inverter and $\bar{v} = D\,V_{dc}$ — tomorrow we tune exactly these loops.

Figure credits

All other diagrams and plots: own drawings, computed for this course (TikZ / pgfplots / circuitikz).

File	Source (Wikimedia Commons)	Author, licence
d3_aliasing.png	File:AliasingSines.svg	Moxfyre, CC BY-SA 3.0
d3_servo_drive_motor.jpg	File:Yeskawa Servo Drive & Motor.jpg	Projuktiponno, CC BY-SA 4.0
d3_servo_drives.png	File:Servo motor drive.png	Tero Kontkanen, CC BY-SA 3.0
d4_kuka_robot.jpg	File:KUKA industrial robotic arm performing "The Beach" by Roy Nachum - 02.jpg	Oleg Yunakov, CC BY-SA 4.0
d1_harmonic_drive.jpg	File:Harmonic Drive AG strain wave gear set.jpg	Pieceofmetalwork, CC BY-SA 4.0
d3_brake_resistor.jpg	File:Turbrake.jpg	Glogger at English Wikipedia, CC BY-SA 3.0
d3_igbt_module_open.jpg	File:CM600DU-24NFH.jpg	Pslawinski, CC BY-SA 3.0
r14_cobot.jpg	File:Cobot.jpg (cropped)	GrowSkills Robotics, CC BY-SA 4.0

References

- K. Ogata, *Modern Control Engineering*, 5th ed., Prentice Hall, 2010.
- G. F. Franklin, J. D. Powell, A. Emami-Naeini, *Feedback Control of Dynamic Systems*, 8th ed., Pearson, 2019.
- G. F. Franklin, J. D. Powell, M. Workman, *Digital Control of Dynamic Systems*, 3rd ed., Addison-Wesley, 1998.
- K. J. Åström, R. M. Murray, *Feedback Systems*, 2nd ed., Princeton Univ. Press, 2021.
- K. J. Åström, T. Hägglund, *Advanced PID Control*, ISA, 2006.
- G. Ellis, *Control System Design Guide*, 4th ed., Butterworth-Heinemann, 2012.
- A. V. Oppenheim, R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.
- M. W. Spong, S. Hutchinson, M. Vidyasagar, *Robot Modeling and Control*, 2nd ed., Wiley, 2020.
- B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer, 2009.
- J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed., Pearson, 2018.
- N. Hogan, “Impedance control: an approach to manipulation,” *ASME J. Dyn. Sys., Meas., Control*, 107(1), 1985.
- N. Mohan, T. M. Undeland, W. P. Robbins, *Power Electronics: Converters, Applications and Design*, 3rd ed., Wiley, 2003.
- R. Krishnan, *Electric Motor Drives: Modeling, Analysis, and Control*, Prentice Hall, 2001.
- IEC 61800-5-1 (adjustable speed drives: safety); IEC 61800-7 (drive profiles).